

Smart HR System

Software Documentation

Version 1.0

Technology:	ASP.NET Core MVC
Database:	SQL Server + EF Core

1. Project Overview

1.1 Project Idea

Smart HR System is a web-based Human Resources Management application built as an academic course project. The goal of this project is to apply practical skills in ASP.NET Core MVC, Entity Framework Core, and SQL Server by building a real-world scenario: managing employees, departments, and attendance in a small organization.

The system allows HR staff to log in, manage employee records, assign departments and roles, track daily attendance, and view a summary dashboard. It is not meant to be a production-grade enterprise system, but it demonstrates solid foundational web development skills.

1.2 Project Objectives

- Apply the MVC design pattern in a real project using ASP.NET Core.
- Practice database design and use EF Core (Code First) for data access.
- Build a working CRUD application with proper relationships between tables.
- Implement basic authentication and authorization using ASP.NET Core Identity.
- Display meaningful summary data using a simple dashboard.

1.3 Main Features

- Employee Management: Add, edit, view, and delete employee records.
- Department Management: Organize employees into departments.
- Role Management: Assign roles to employees (e.g., Manager, Developer, HR).
- Attendance Tracking: Record daily check-in and check-out times per employee.
- Dashboard Statistics: View counts, summaries, and quick status overviews.
- Secure Login: Only authenticated users can access the system.

2. System Features

This section describes each feature of the system in brief detail.

2.1 Employee Management

The core feature of the system. HR staff can add new employees with details such as name, email, phone number, hire date, and assigned department and role. They can also update an employee's information or remove them from the system. Each employee is listed in a searchable, paginated table.

2.2 Department Management

Departments represent the different teams in the organization (e.g., Engineering, Sales, HR). Admins can create new departments, rename them, or delete ones that are no longer in use. Each department can have multiple employees assigned to it.

2.3 Role Management

Roles define job titles or positions within the company, such as Manager, Developer, Designer, or Accountant. These are simple labels stored in the database and assigned to employees. This is separate from ASP.NET Core Identity roles, which control system access.

2.4 Attendance Tracking

HR staff can record a daily attendance entry for each employee. Each record stores the employee, date, check-in time, check-out time, and an optional note (e.g., Late, On Leave). A simple list view shows all attendance records, filterable by date or employee.

2.5 Dashboard Statistics

The home page after login shows a summary of the system data: total number of employees, number of departments, employees checked in today, and any recent activity. This gives HR staff a quick overview without navigating to each section.

3. Functional Requirements

3.1 Login

1. The system must allow users to log in using their email and password.
2. If the credentials are incorrect, the system displays an error message.
3. Logged-in users are redirected to the dashboard.
4. Users can log out from any page using the navigation bar.
5. All pages (except the login page) require authentication.

3.2 Add / Edit / Delete Employees

6. A logged-in user can view the full employee list on the Employees page.
7. A user can add a new employee by filling in a form (name, email, phone, hire date, department, role).
8. All required fields must be filled in before the form can be submitted.
9. A user can click Edit on any employee to update their information.
10. A user can delete an employee after confirming a prompt.
11. Deleting an employee also removes their attendance records.

3.3 Manage Departments

12. A user can view all existing departments in a list.
13. A user can add a new department by providing a name and optional description.
14. Department names must be unique in the system.
15. A user can edit a department's name or description.
16. A department cannot be deleted if it has employees assigned to it.

3.4 Record Attendance

17. A user can open the attendance page and record an entry for any employee.
18. Each attendance record requires: employee, date, check-in time, and status.
19. Check-out time and notes are optional.
20. An employee cannot have more than one attendance record per day.
21. A user can view attendance records filtered by date or employee.

3.5 View Reports / Dashboard

22. The dashboard displays the total number of employees.
23. The dashboard shows the total number of departments.
24. The dashboard shows how many employees checked in today.
25. A recent activity section lists the last few changes made in the system.

4. Database Design

The database is designed using EF Core Code First. Below are the five tables used in the system, with their columns, data types, and key information.

4.1 Employees

Column	Data Type	Notes
Id	int	Primary Key, auto-increment
FirstName	nvarchar(100)	Required
LastName	nvarchar(100)	Required
Email	nvarchar(200)	Required, unique
Phone	nvarchar(20)	Optional
HireDate	date	Required
DepartmentId	int	Foreign Key -> Departments.Id
RoleId	int	Foreign Key -> Roles.Id
CreatedAt	datetime	Set automatically on insert

4.2 Departments

Column	Data Type	Notes
Id	int	Primary Key, auto-increment
Name	nvarchar(100)	Required, unique
Description	nvarchar(500)	Optional

4.3 Roles

Column	Data Type	Notes
Id	int	Primary Key, auto-increment
Title	nvarchar(100)	Required, unique (e.g., Manager, Developer)
Description	nvarchar(500)	Optional

4.4 AttendanceRecords

Column	Data Type	Notes
Id	int	Primary Key, auto-increment
EmployeeId	int	Foreign Key -> Employees.Id
Date	date	Required
CheckIn	time	Required
CheckOut	time	Optional
Status	nvarchar(50)	e.g., Present, Late, On Leave
Notes	nvarchar(500)	Optional

4.5 Users

Column	Data Type	Notes
Id	nvarchar(450)	Primary Key (GUID), managed by Identity
UserName	nvarchar(256)	Login username
Email	nvarchar(256)	User email address
PasswordHash	nvarchar(max)	Hashed password, managed by Identity
CreatedAt	datetime	Custom field added to ApplicationUser

4.6 Table Relationships

- **Department → Employees:** One Department has many Employees (One-to-Many).
- **Role → Employees:** One Role has many Employees (One-to-Many).
- **Employee → AttendanceRecords:** One Employee has many AttendanceRecords (One-to-Many).

Users (the login system) are managed separately by ASP.NET Core Identity and are not directly linked to the Employees table in this version.

5. Entity Relationship Diagram (ERD)

The following is a text-based description of the ERD for the system:

```
[ Departments ]
PK  Id
    Name
    Description

[ Roles ]
PK  Id
    Title
    Description

[ Employees ]  <-- belongs to Departments (FK: DepartmentId)
PK  Id
    FirstName, LastName, Email, Phone, HireDate
FK  DepartmentId --> Departments.Id
FK  RoleId       --> Roles.Id

[ AttendanceRecords ]  <-- belongs to Employees (FK: EmployeeId)
PK  Id
    Date, CheckIn, CheckOut, Status, Notes
FK  EmployeeId  --> Employees.Id

[ Users ]  (ASP.NET Core Identity - standalone)
PK  Id (GUID)
    UserName, Email, PasswordHash, CreatedAt
```

Relationships summary:

- Departments (1) -----< Employees (many)
- Roles (1) -----< Employees (many)
- Employees (1) -----< AttendanceRecords (many)
- Users table is standalone (managed by Identity).

6. System Architecture

The system follows the standard MVC (Model-View-Controller) pattern provided by ASP.NET Core. Here is how the main layers are organized:

6.1 ASP.NET Core MVC (Presentation + Application Layer)

- Models: C# classes that represent the data (e.g., Employee, Department). These are also the EF Core entities.
- Views: Razor pages (.cshtml) that render the HTML output to the browser using Bootstrap for styling.
- Controllers: Handle incoming HTTP requests, call the data layer, and return the appropriate view or redirect.

6.2 Entity Framework Core (Data Access Layer)

- The project uses EF Core with Code First, meaning the database is generated from C# entity classes.
- A single ApplicationDbContext class inherits from DbContext and registers all entities as DbSet properties.
- Migrations are used to create and update the database schema.

6.3 SQL Server (Database)

- SQL Server is the database engine used to store all application data.
- EF Core handles all SQL queries behind the scenes using LINQ.
- The connection string is stored in appsettings.json.

6.4 Request Flow

26. User opens a page in the browser (e.g., /Employees/Index).
27. ASP.NET Core routes the request to the correct controller action.
28. The controller calls the ApplicationDbContext to fetch or save data.
29. EF Core translates the LINQ query to SQL and queries SQL Server.
30. The controller passes the result to a Razor view.

7. Future Improvements

The following enhancements can be added in future versions of the project:

- Employee Self-Service Portal: Allow employees to log in and view their own attendance history without needing HR to do it for them.
- Leave Request Management: Add a module where employees can submit leave requests and HR staff can approve or reject them.
- Email Notifications: Send automated emails when an employee is added, a leave request is submitted, or an attendance issue is flagged.
- Advanced Reporting: Add filter-based reports for attendance (e.g., absences by month) with the option to export to PDF or Excel.
- Role-Based Access Control: Differentiate between an Admin user (full access) and a regular HR user (limited access) within the system.
- Payroll Integration: Add a basic salary and payroll calculation module based on attendance and leave records.
- REST API Layer: Expose the system data through a separate API so it can be consumed by a mobile app or a frontend JavaScript framework.

8. Conclusion

Smart HR System is a straightforward project that demonstrates how to build a functional web application using ASP.NET Core MVC, Entity Framework Core, and SQL Server. It covers the key concepts of web development: MVC architecture, database design with relationships, CRUD operations, and basic authentication.

Project Members

Mohamed Mahmoud Mahdy

Mahmoud shams eldin

Hadeer Mohamed

Hager Magdy

Motasem Mohamed